

Hermes Creative Workstation — investigação tecnológica, arquitetural e de código open source

Pesquisa consolidada em **9 de outubro de 2026**, com auditoria direta de repositórios, documentação oficial, código-fonte, branches, forks, issues, releases e do repositório real do Hermes Work. A conclusão principal mudou de forma material em relação às hipóteses iniciais: **há hoje um candidato que se aproxima mais da visão “humano + agente + código sobre o mesmo projeto editável” do que Penpot, OpenCut ou mesmo OpenReel: o `diffusionstudio/editor`**. Ele ainda é jovem demais para ser adotado cegamente, mas sua arquitetura merece ser o principal objeto de validação.

Minha recomendação não é, contudo, “trocar Hermes pelo Diffusion Studio”. O resultado mais forte da investigação é uma **arquitetura híbrida com uma única autoridade de composição**, na qual o Hermes conserva seu runtime, Task Compiler, Control Plane, ArtifactStore e Experience Compiler; reaproveita um núcleo editor/runtime já existente; e conecta motores especializados — Three.js, FFmpeg/MediaBunny, Blender e IA — por adaptadores tipados. Essa abordagem evita tanto a reescrita de um editor inteiro quanto o problema quase insolúvel de manter quatro documentos soberanos sincronizados. A própria arquitetura atual do Hermes já proíbe a criação de um segundo runtime, registry ou project database soberano, o que é consistente com essa direção.

Mapa tecnológico

Design e edição visual bidimensional

O espaço 2D se divide em dois grupos muito diferentes: **editores completos**, como Penpot, Graphite e Excalidraw, e **bibliotecas de cena/renderização**, como Fabric, Konva, SVG.js, Paper.js e PixiJS. Confundir esses grupos leva a subestimar anos de trabalho de UI, seleção, snapping, texto, edição de nós, undo/redo, inspector e persistência.

Projeto	Tipo	Pontos fortes para Hermes	Limitação central	Licença / URL
Penpot	Editor completo	Melhor candidato open source para design vetorial/UI sofisticado; SVG/CSS/HTML/JSON; componentes, layouts, plugins e MCP	Não é NLE nem 3D; transformar o núcleo em After Effects/Premiere exigiria alterações profundas	MPL-2.0 — https://github.com/penpot/penpot ¹

Projeto	Tipo	Pontos fortes para Hermes	Limitação central	Licença / URL
Fabric.js	Biblioteca	Canvas interativo, objetos serializáveis, grupos, texto editável, SVG import/export, filtros e clipping	É necessário construir toda a aplicação/editor ao redor; SVG não é round-trip perfeito	MIT — https://github.com/fabricjs/fabric.js ²
Konva	Biblioteca	Scene graph Canvas, transformadores, eventos, drag, React integration, export	Modelo gráfico mais simples que um editor vetorial profissional	MIT — https://github.com/konvajs/konva ³
SVG.js	Biblioteca	Manipulação SVG direta, boa interoperabilidade e animação básica	Não oferece editor nem modelo completo de design	MIT — https://github.com/svgdotjs/svg.js ⁴
Paper.js	Biblioteca vetorial	Path/Bézier e geometria vetorial são sua principal vantagem	UI/editor, timeline e vídeo ficariam por conta do Hermes	https://github.com/paperjs/paper.js ⁵
PixiJS	Renderer	Excelente performance 2D/ WebGL/WebGPU e composição	É renderer, não ferramenta de desenho ou documento de design	MIT — https://github.com/pixijs/pixijs ⁶
Excalidraw	Editor incorporável	React, JSON aberto, colaboração, export PNG/SVG, MCP recente	Sem profundidade para substituir Illustrator/Figma	MIT — https://github.com/excalidraw/excalidraw ⁷
tldraw	SDK/editor	Arquitetura e extensibilidade excelentes	A licença atual do SDK não é uma licença open source permissiva; produção possui requisitos próprios	https://github.com/tldraw/tldraw ⁸
Graphite	Editor procedural	Rust/WASM, grafo procedural, visão ambiciosa cobrindo design e arte	Módulos de criação e motion ainda evoluem; risco alto para curto prazo	MIT/Apache-2.0 — https://github.com/GraphiteEditor/Graphite ⁹

Conclusão para 2D: Penpot é tecnicamente muito superior a Fabric/Konva como **editor de design pronto**, mas Fabric e Konva são mais fáceis de incorporar a uma aplicação React existente. Construir “um Penpot próprio em Konva” seria uma grande quantidade de trabalho redundante. A decisão só muda se o Creative Workstation precisar de um vocabulário 2D menor que o de Figma/Illustrator.

Motion graphics e animação

Aqui também há uma separação importante entre **engines de animação** e **sistemas de autoria**.

O projeto mais interessante encontrado foi **Diffusion Studio Editor**, cujo documento é código SolidJS/JSX e cujo runtime monta esse documento diretamente em uma ECS. Alterações feitas pelo humano no canvas ou timeline são escritas de volta no código que as originou; alterações feitas pelo agente no código aparecem novamente no editor. Isso resolve de forma direta um dos problemas mais difíceis do Hermes: a manutenção de identidade e editabilidade após operações de IA.

O seu modelo de keyframes é explícito: `<keyframeTrack>` controla uma propriedade, keyframes possuem tempo/valor/easing, suportam cubic Bézier, springs e steps, e cada elemento/keyframe mantém `id` para round-trip entre source e editor.

Outros projetos mantêm papéis mais específicos:

Tecnologia	Avaliação para Hermes
Diffusion Studio Editor	Melhor arquitetura encontrada para agente ↔ código ↔ UI ↔ timeline; ainda jovem e incompleto em design vetorial. MPL-2.0. https://github.com/diffusionstudio/editor
OpenReel Motion Creator	Motion editor já conectado ao projeto de vídeo, keyframes, layers, shaders, partículas e <code>scene3d</code> ; muito relevante para resultados imediatos.
Remotion	Excelente renderização determinística React/code-first, mas não fornece sozinho um editor universal; sua licença comercial atual precisa de análise específica para produtos que automatizam/renderizam vídeo. ¹⁰
Revideo	Fork MIT de Motion Canvas orientado a vídeo programático/headless; bom motor, não um editor final. https://github.com/midrender/revideo ¹¹
Motion Canvas	Excelente TS code-first para animações vetoriais/explicativas; editor de preview, não NLE generalista. MIT. ¹²
Theatre.js	Ótima referência para sequencer, keyframes e graph editor, inclusive Three.js; entretanto <code>core</code> e <code>Studio</code> têm licenças diferentes e o Studio traz copyleft mais forte. ¹³
Motion	Biblioteca moderna React/JS/Vue com timeline, springs, layout e gestos; apropriada para animações internas, não como documento/timeline autoritativa. A versão do pacote encontrada é 14.0.0. ¹⁴
GSAP	Engine de animação extremamente capaz e possui material oficial para agentes, mas o repositório principal usa a licença padrão do GreenSock, não deve ser classificado automaticamente como MIT por causa do repositório separado de skills. ¹⁵

Tecnologia	Avaliação para Hermes
Anime.js	Engine procedural compacta; útil dentro de superfícies programáveis, não substitui timeline/editor. ¹⁶
Lottie	Excelente formato/runtime de distribuição de animação AE; não é um documento criativo universal e possui limitações de fidelidade para determinados recursos de AE. ¹⁷

A descoberta particularmente interessante é que o Diffusion já demonstra **anime.js dirigido pelo playhead**, raw WebGPU, WGSL e Three.js como superfícies dentro da mesma composição.

Três dimensões

Para o editor incorporável, **Three.js continua sendo a escolha mais pragmática**. É MIT, suporta WebGL/WebGPU, glTF, skeletal animations, morph targets, materiais, câmeras e luzes. O ecossistema `GLTFLoader` / `GLTFExporter` torna GLB/glTF o formato natural de interoperabilidade. ¹⁸

`React Three Fiber` é especialmente atraente no Hermes porque traduz Three para um modelo declarativo no ecossistema React; portanto serve melhor ao Desktop existente que uma reescrita independente de UI. ¹⁹

Babylon.js também é tecnicamente excelente, Apache-2.0, e possui engine WebGL/WebGPU completa. **PlayCanvas** merece ser lembrado como alternativa pouco discutida: seu engine é MIT e foi concebido como sistema de cena completo para aplicações 3D web. Entretanto, adotar outro scene graph inteiro ao lado de Three cria mais conversão sem um benefício demonstrado para o Hermes. ²⁰

Blender deve ser tratado como DCC externo, não como editor incorporado. Suas APIs Python e modo background/headless são muito valiosos para modelagem, rigging, baking e renderização. Incorporar a aplicação Blender inteira no shell Electron seria arquiteturalmente e juridicamente muito mais complexo, além de não resolver a unificação do documento. ²¹

O melhor padrão encontrado é exatamente aquele já demonstrado no Diffusion: uma camada/surface recebe um renderer Three.js e participa da mesma linha temporal da composição.

Vídeo e mídia

FFmpeg continua indispensável, mas como processador/exportador, não como modelo de edição. Seus filtergraphs conseguem representar pipelines complexos, porém a UI, identidade de clips, undo, keyframes e projeto pertencem a outra camada. A licença de uma distribuição concreta depende de como o FFmpeg foi compilado: o conjunto padrão é LGPL, enquanto a ativação de componentes GPL muda as obrigações da build. ²²

MediaBunny é uma peça muito atraente para a camada web/desktop porque mantém o processamento em TypeScript/WebCodecs, cobre containers e operações de mídia e pode reduzir muito a dependência de subprocessos durante preview. O próprio Diffusion Studio novo usa MediaBunny no encoder. ²³

MLT e GStreamer são opções maduras para construir NLEs nativos. MLT foi criado exatamente como engine de composição multitrack; GStreamer Editing Services possui sua própria timeline/layer/clip abstraction. Entretanto, adotar qualquer um como centro do Hermes significaria reconstruir uma

grande camada de UI e bridges C/C++/Rust, enquanto OpenReel/Diffusion já disponibilizam documentos e editors web. ²⁴

Assim:

preview interativo: WebCodecs + MediaBunny + Canvas/WebGPU;

export e fallback: FFmpeg desktop;

MLT/GStreamer: somente se testes demonstrarem que MediaBunny/WebCodecs/FFmpeg não conseguem atender a casos profissionais necessários.

Inteligência artificial local

Para IA, a conclusão é não colocar modelos dentro do documento. **O documento deve guardar resultados editáveis e proveniência; modelos devem ser capacidades externas reproduzíveis.**

SAM 2 possui APIs para segmentação em imagem e vídeo e seu pacote identifica Apache-2.0, sendo uma excelente base para masks/rotoscopia assistida. ²⁵

`rembg` continua uma opção simples MIT para background removal; `faster-whisper` é uma solução local eficiente para transcrição; Demucs é tecnicamente forte para separação de stems, mas código, checkpoints e dados precisam ser auditados separadamente. ²⁶

Para tracking, **não adotaria CoTracker como dependência padrão comercial sem resolver licença:** o repositório oficial informa que a maior parte está sob CC-BY-NC. ²⁷

Para interpolação, o ecossistema RIFE continua útil; a variante Practical-RIFE afirma licença MIT também para os modelos disponibilizados por ela, mas isso precisa ser pinado por versão/checkpoint no admission process do Hermes. ²⁸

Robust Video Matting é tecnicamente excelente para vídeo humano, inclusive com processamento temporal em tempo real, mas seu código foi relicenciado como GPL-3.0; portanto não é a primeira escolha para um núcleo distribuído permissivo. ²⁹

Para geração, **ComfyUI deve ser um backend externo de workflows de IA, não o documento criativo do Hermes.** O projeto atual se posiciona como engine modular para imagem, vídeo, 3D e áudio. ³⁰ O fork OpenReel-ComfyUI encontrado demonstra que OpenReel pode de fato ser incorporado por iframe/postMessage e receber mídia de workflows ComfyUI, incluindo modificações de routing, embedding e composição multiclips. Isso é uma evidência útil de incorporabilidade — mas não o transforma automaticamente em melhor fork geral para Hermes.

Para geração 3D, TRELIS é tecnicamente interessante e o repositório principal declara MIT para a maior parte do código/modelos, com submódulos a auditar. TRELIS.2 também apresenta licença MIT no repositório, mas há questões atuais sobre dependências e suporte não-CUDA, de modo que é candidato de laboratório, não dependência obrigatória. ³¹ Hunyuan3D não deve ser classificado simplesmente como “open source permissivo”: seus próprios termos contêm condições/restrições específicas. ³²

Repositórios e forks

A descoberta principal: Diffusion Studio Editor

Repositório oficial:

`https://github.com/diffusionstudio/editor`

Em 9 de outubro de 2026, o repositório era TypeScript, MPL-2.0, havia sido criado em julho de 2026 e continuava recebendo commits em outubro. Isso é ao mesmo tempo uma evidência de desenvolvimento muito ativo e um sinal de **maturidade cronológica baixa**.

O que o torna especialmente relevante não é o número de features, e sim a arquitetura:

projeto = source code = documento.

O source é Solid JSX. O compilador injeta IDs estáveis; o reconciler Solid monta os elementos diretamente em uma ECS; e alterações feitas visualmente são reescritas na propriedade do elemento que originalmente as produziu.

A estrutura do monorepo é muito boa para reutilização seletiva:

Pacote	Função
<code>apps/web</code>	editor visual
<code>apps/desktop</code>	shell Electron
<code>apps/cli</code>	CLI para o MCP/runtime
<code>packages/runtime</code>	ECS, ações, sistemas, media decode e capture; explicitamente desacoplado de DOM/Solid
<code>packages/reconciler</code>	JSX compilado → entities
<code>packages/jsx</code>	linguagem/autoria
<code>packages/assets</code>	manifest, hashing, probing e resolução
<code>packages/encoder</code>	export offline com MediaBunny
<code>packages/koota-solid</code>	bindings ECS/Solid

Essa decomposição está documentada no próprio repositório.

O projeto também possui MCP/CLI orientado a agentes, com ferramentas para `probe`, frame grab, filmstrip, waveform, transcript, inspection e capture. O ponto importante é que o agente não cria uma renderização descartável: ele modifica o mesmo source que o humano abre no editor.

Problemas encontrados: o vocabulário nativo 2D atual não contém uma ferramenta equivalente a um editor Bézier completo. Os elementos documentados incluem rect, text, image, video, audio, captions,

HTML, surfaces, paints, effects e keyframes, mas não um `path` arbitrário equivalente ao Illustrator/ Penpot; a documentação também afirma que o layout engine ainda não está disponível.

Portanto, Diffusion Studio é hoje **um editor de composição/motion/vídeo extraordinariamente alinhado à IA, não um substituto completo de Penpot.**

OpenReel

Repositório:

<https://github.com/Augani/openreel-video>

Em 9 de outubro, o repositório era TypeScript/MIT, criado em janeiro de 2026 e ainda ativo.

A auditoria do código mostra que OpenReel é muito mais que um README de “CapCut clone”. O `Project` real agrega:

`mediaLibrary`, `timeline`, `textClips`, `shapeClips`, `svgClips`, stickers, adjustment layers, masks, multicam, compound clips, nested instances, `motionCompositions`, `motionInstances`, shaders e `creation`.

O serializer possui versionamento (`1.2.0` no ref auditado), minimum reader version, capabilities, normalização das estruturas Motion e validação de referências de mídia.

Isso muda uma conclusão importante: **OpenReel não é apenas “vários aplicativos escondidos atrás da mesma navbar”**. Há realmente uma estrutura `Project` superior que contém timeline, motion, overlays e outros subsistemas.

Ao mesmo tempo, não é um scene graph totalmente uniforme. A enumeração de campos do `Project` demonstra que texto, shapes, timeline, Motion e Creation ainda são subsistemas semânticos diferentes. Logo, eu o classificaria como **documento federado único**, não como uma única ontologia universal.

Motion Creator tem tipos nativos para:

text, shape, image, video, group, null, nested composition, adjustment, particle e `scene3d`; efeitos, máscaras, track mattes e expressions; modelos glTF/GLB; materiais físicos; câmera e luzes; objetos múltiplos; clips de animação glTF; e até baked/deforming mesh frames.

Isso faz OpenReel **muito mais avançado em 3D integrado ao motion do que seria correto inferir apenas da tela principal.**

Há, porém, sinais explícitos de limites. A documentação de rigging do próprio projeto reconhece que colocar GLB estático não resolve personagens articulados e que workflows de DCC exigem pipeline externo.

O Agent Guide afirma que as ferramentas de agente operam alterações reais do projeto — mídia, timeline, trim, transform, color grade, áudio, shapes, legendas, keyframes e transições — e há um router separado para operações Motion.

Isso atende um requisito crucial do Hermes: **a IA pode modificar estruturas que continuam editáveis manualmente.**

Forks de OpenReel

A busca de forks não revelou um fork genérico claramente superior ao upstream. A exceção arquitetural interessante foi:

```
https://github.com/WASasquatch/openreel-video-comfyui
```

Ele adiciona embedding em ComfyUI, base paths, comunicação `postMessage`, modo embedded, theme sync e correções de multi-clip/blend durante playback. O próprio fork documenta claramente o delta em relação ao upstream.

Esse fork **não deve substituir o upstream como base geral**, mas prova duas coisas úteis:

1. OpenReel tolera incorporação em uma aplicação maior.
2. Um boundary por mensagens para enviar resultados de IA ao editor funciona na prática.

Penpot e Penpot Motion

Upstream:

```
https://github.com/penpot/penpot
```

Penpot é o sistema mais maduro da pesquisa para design vetorial/UI e mantém MPL-2.0. A arquitetura atual inclui frontend/backend/common, plugins, MCP, exporter, media processing e renderer WASM/Skia em evolução. ³³

O MCP atual consegue inspecionar e modificar elementos de design e se comunica com o ambiente Penpot por plugin/WebSocket. Isso é uma excelente integração para Hermes, mas o poder de execução dentro do ambiente de plugin torna obrigatório aplicar a autoridade e os efeitos pelo Control Plane do Hermes, não entregar execução irrestrita ao modelo. ³⁴

O fork solicitado foi confirmado:

```
https://github.com/girafic/penpot
```

branch:

```
girafic-penpot-timeline-animation
```

Em 8 de outubro de 2026, a branch apontava para `ce40a985d3286d30a2af88e21dcd12c832dedd8`; o último commit auditado corrigia sincronização entre inicialização do renderer WASM, começo da animação e áudio. Portanto, **não é apenas uma branch abandonada ou um mockup.**

A arquitetura anunciada pela implementação é particularmente inteligente: shapes Penpot continuam shapes Penpot; a timeline é adicionada por página/board; tracks apontam para IDs de layers; o playback converte animação em modificadores Penpot normais. Também há auto-keyframe, motion

paths, markers, export animado e API/plugin/MCP para timeline. A própria apresentação do fork o classifica como experimental e aponta recursos ainda ausentes, como morphing completo de path e vários casos avançados de Lottie. ³⁵

Veredito do fork: é arquiteturalmente melhor que o Penpot original **para estudar como anexar motion a um documento vetorial existente**, mas não é superior como base de produção geral. Não resolve NLE, áudio profissional ou 3D; e o custo de acompanhar o enorme upstream Penpot permanece.

OpenCut

Repositório:

<https://github.com/OpenCut-app/OpenCut>

OpenCut precisa ser avaliado com muito cuidado porque documentação de futuro e implementação presente diferem significativamente. O README atual fala de um novo core Rust, plugin-first, editor API, MCP, scripting e headless — mas esses itens fazem parte da **reescrita**, não devem ser tratados como capacidades prontas. ³⁶

Há, no entanto, código real muito interessante. O monorepo atual contém Cargo workspace/crates e aplicações.

A release 0.3.0 trouxe uma direção tecnicamente muito boa para um editor:

- tempo em integer ticks (`MediaTime`) em vez de floats;
- `FrameRate` racional;
- compositor Rust/wgpu compilado para WASM;
- efeitos/masks/GPU em crates;
- graph editor para curvas/keyframes;
- melhorias de ripple, track ordering e projeto. ³⁷

Esse modelo de tempo é algo que **vale copiar como conceito** mesmo que OpenCut não seja adotado.

Por outro lado, o futuro desktop Rust/GPUI ainda estava muito mais inicial do que o produto clássico, e a API/MCP prometida não deve receber pontos como feature implementada. ³⁸

Situação real no Hermes Work

Este ponto é decisivo: o Creative Workstation **ainda é PLANNED/DOCS ONLY no repositório**, e o roadmap já descreve provas incrementais para React/SVG, FFmpeg, Remotion, Penpot e Three.

O `SOURCE_MATRIX.md` já contém uma excelente regra metodológica: referências externas são leads, `NV` significa “not verified”, e README não equivale a approval de runtime. O documento também já distingue corretamente Penpot core MPL, AI kit, Remotion, FFmpeg, Blender, Graphite e outros.

Mais importante, o Hermes **já tem os componentes que deveriam controlar o Creative Workstation**.

`TaskCompiler` compila trabalho estruturado para a via de execução existente, aplica constraints, receitas, `OperationalCapabilities` e verificadores; ele deliberadamente não é mais um planner/model.

`OperationalCapability` já possui versão, schemas, efeito, route, scope, preconditions, postconditions, verifier contract, dependencies, provenance, fingerprints, validation evidence, lifecycle e drift.

E o Experience Compiler já existe como código concreto, com compiler, causal analysis, corpus, generalization, hierarchical compilation, lifecycle, promotion e validation.

Portanto, criar um “Creative Agent Runtime”, um segundo registry ou um novo workflow engine seria **duplicação arquitetural**, não progresso.

Análise arquitetural

A questão central não é qual renderer escolher

O problema mais difícil não é Canvas versus SVG versus WebGPU. É este:

Quem é a autoridade sobre identidade, tempo, propriedades e relações entre objetos?

Se Penpot controla `shape-42`, Three controla `mesh-7`, um NLE controla `clip-19` e Remotion controla um componente React separado, é necessário construir um mecanismo de sincronização em quatro direções:

`Penpot ↔ timeline ↔ Three ↔ render source`.

Isso parece modular, mas cada sistema tem semântica diferente para:

- parent/child;
- transform;
- masks;
- coordinate system;
- text metrics;
- frame rate e tempo;
- keyframes;
- effects;
- nested compositions;
- asset lifecycle;
- undo/redo;
- stable IDs.

O risco não é apenas escrever adapters. O risco é **não existir uma única resposta para “qual é o estado verdadeiro?”**.

Por isso a alternativa “quatro aplicativos especializados sincronizados” é a pior para a ambição de longo prazo, apesar de parecer rápida no começo.

A arquitetura superior encontrada

A melhor solução é uma variante da opção híbrida:

um documento/composition authority + render adapters especializados.

Não é necessário que o documento universal represente internamente cada vértice de um arquivo Blender ou cada construct de Penpot. Ele precisa representar universalmente aquilo que cruza as disciplinas:

```
Project
├─ Assets
│   ├── Image / Video / Audio
│   ├── SVG / Vector
│   ├── GLTF / 3D Scene
│   └─ Native external source
├─ Compositions
│   ├── Node
│   │   ├── stable id
│   │   ├── type
│   │   ├── hierarchy
│   │   ├── spatial transform
│   │   ├── temporal range
│   │   ├── properties
│   │   ├── masks/effects
│   │   └─ animation tracks
│   └─ nested composition
├─ Timeline
├─ Markers
├─ Audio
├─ Color / render settings
└─ Provenance / versions / capabilities
```

O objeto de design importado de Penpot pode continuar tendo um Penpot-native source; o objeto Blender pode continuar apontando para `.blend` ou GLB; o vídeo continua arquivo de mídia. O documento universal é responsável por **composição e editabilidade compartilhada**, não por converter sem perda todo formato existente.

Isso é compatível com a arquitetura atual do Hermes, cujo Creative Project Manifest já reconhece fontes nativas e explicitamente não promete conversão lossless entre Penpot, React/SVG e Three.

Minha alteração recomendada à ideia atual é tornar esse manifest mais que um índice passivo: **ele deve evoluir para o contrato canônico de composição**, ou adotar um formato com essa propriedade já existente.

Por que Diffusion Studio é arquiteturalmente importante

O Diffusion Studio oferece uma resposta inédita entre os candidatos estudados:

o próprio source/AST é a autoridade; a ECS é a representação runtime.

A pipeline documentada é:

```
stamp stable IDs → compile → evaluate → mount into ECS → resolve media → live graph.
```

Preview, capture e export voltam a executar o mesmo módulo.

É uma resposta muito mais forte ao problema humano/IA do que “LLM edita JSON que depois tentamos converter de volta para a UI”.

O padrão que merece ser adotado é:

stable identity + typed document + deterministic evaluator + round-trip source ↔ visual editor.

Isso não significa necessariamente manter JSX para sempre. Uma possível evolução Hermes pode usar um AST/IR tipado como forma canônica e gerar JSX apenas como authoring language. Mas começar inventando um IR próprio antes de testar o Diffusion seria repetir trabalho que já existe.

Preview e export devem usar o mesmo avaliador

Outra regra arquitetural obrigatória é evitar:

```
preview renderer A + export renderer B
```

quando ambos reinterpretam independently o documento.

O ideal é:

```
Creative document
  ↓
Frame evaluator(t)
  ↓
Render adapters
  2D / video / 3D / shader
  ↓
RGBA + audio
  ↓
Preview
  ou
Encoder
```

Assim, a diferença entre preview e export é principalmente qualidade/resolução/codec, não semântica.

OpenCut oferece uma segunda ideia valiosa: o clock deve usar **tempo inteiro/racional**, não floats de JavaScript acumulados, para que frames, trim e áudio tenham endereçamento determinístico. ³⁷

Linguagens e runtimes

Camada	Recomendação	Motivo
UI/editor	TypeScript + React , preservando o Hermes Desktop	Evita reescrita e tem melhor integração com scene/render libraries existentes
Documento e operações	TypeScript schemas/AST + JSON serializável quando aplicável	Mesmo tipo compartilhado entre editor, agent tools e renderer
Hot paths determinísticos	Rust → WASM	Excelente para timeline math, geometry, masks, compositor e kernels CPU/GPU; OpenCut demonstra essa arquitetura ³⁷
IA/orquestração	Python	Hermes já o possui; ecossistema de SAM, Whisper, Blender e PyTorch
Mídia	MediaBunny/WebCodecs + FFmpeg	Web-native no caminho interativo; FFmpeg para compatibilidade/export ³⁹
3D	Three.js / R3F	MIT, WebGL/WebGPU, glTF e melhor fit com o Desktop web ⁴⁰
GPU	WebGPU/WGSL , mantendo fallback	Excelente para shaders/effects modernos; Diffusion já demonstra shader WGSL e raw WebGPU
Interchange vetorial	SVG	Ecossistema maduro e compatível com Penpot/Fabric; não deve ser o documento temporal inteiro
C/C++	Somente via projetos externos maduros	FFmpeg/Blender/GStreamer já cobrem esse território; não há justificativa encontrada para novo núcleo Hermes em C++

Licenciamento

MIT/Apache/BSD continuam ideais para kernels internos.

MPL-2.0 é aceitável, não um motivo suficiente para rejeitar Diffusion ou Penpot. Mozilla descreve MPL como copyleft no nível de arquivo: modificações em arquivos MPL distribuídos permanecem sujeitas à MPL, enquanto a licença foi desenhada para permitir combinação com código sob outras licenças, inclusive proprietário, com menos propagação que GPL. ⁴¹

Isso torna possível manter um boundary relativamente limpo:

```
Hermes proprietary/permissive code
|
typed adapter/API
```

As alterações nos arquivos MPL precisariam ser tratadas conforme a MPL quando distribuídas.

LGPL é geralmente mais amigável para uso de biblioteca dinâmica/isolada que GPL, embora a forma específica da combinação continue importante. ⁴²

GPL merece isolamento por processo/CLI quando a estratégia comercial não deseja um derived combined work sob GPL; a própria GNU FAQ trata linking como um dos pontos críticos de compatibilidade. ⁴³

AGPL traz risco adicional em execução como serviço: a licença contempla oferta do source da versão modificada aos usuários que interagem remotamente com o servidor. ⁴⁴

E quatro licenças devem ser auditadas separadamente em IA/mídia:

código ≠ weights ≠ dataset ≠ codec/build.

Isso é particularmente importante para FFmpeg, Demucs, CoTracker, TRELIS/Hunyuan e qualquer modelo de geração. ⁴⁵

Auditoria dos finalistas

Diffusion Studio

Resultado da auditoria: melhor correspondência arquitetural com a visão Hermes, mas ainda não suficientemente maduro para decisão irreversível.

Capacidades comprovadas por código/documentação do projeto:

- vídeo, áudio, texto, rects, gradients, effects, masks e blend;
- timeline;
- keyframes;
- transitions;
- code-driven animation;
- source↔canvas round-trip;
- assets;
- headless runtime;
- encoder;
- MCP/CLI;
- surfaces programáveis;
- Three.js;
- raw WebGPU/WGSL;
- agente modifica o mesmo documento editável.

Capacidades ainda insuficientes:

- pen tool profissional;
- arbitrary path/Bézier editing como first-class element;

- node editor vetorial;
- layout engine maduro;
- first-class 3D scene graph editável com inspector completo;
- longa história de produção/compatibilidade.

O maior risco é temporal: o repositório só existe desde julho de 2026.

OpenReel

Resultado: melhor combinação de funcionalidades já existentes + licença permissiva + UI convencional.

O projeto já está mais perto de entregar “vídeo utilizável amanhã” do que qualquer estratégia de construção própria. O modelo superior de projeto realmente une video timeline, overlays, Motion e Creation, e o Motion layer vocabulary inclui `scene3d`.

Seu ponto fraco de longo prazo é que o documento cresce por agregação de subsistemas:

```

timeline
textClips
shapeClips
svgClips
motionCompositions
creation
...

```

em vez de todos serem a mesma espécie de node/property/time entity.

Isso não o torna ruim. Torna **mais provável que o custo de unificação cresça com cada novo domínio**.

Penpot Motion

Resultado: experimento tecnicamente sério, não production foundation.

A ideia de guardar timeline separadamente dos shapes e aplicar animação como modifiers normais é provavelmente a melhor maneira de adicionar motion sem destruir a semântica Penpot. ³⁵

Mas migrar Hermes para um fork Penpot profundamente modificado criaria uma responsabilidade permanente sobre uma codebase Clojure/ClojureScript/Rust muito grande, ao mesmo tempo em que ainda seria necessário adicionar:

- NLE;
- decoding/encoding;
- waveform/audio;
- 3D scene layer;
- media cache;
- deterministic render/export.

O saldo de trabalho não é favorável.

OpenCut

Resultado: referência para kernels, não base recomendada hoje.

A arquitetura Rust/WGPU, tempo inteiro e rational framerate merece ser estudada. A promessa futura de MCP/API/headless é exatamente alinhada ao Hermes. Mas promessa não vale nota de feature implementada; a reescrita ainda não atingiu paridade suficiente para justificar apostar o Workstation nela. 38

Estado dos testes solicitados

Não atribuo **PASS** a nenhum dos seis testes.

Tentei iniciar a etapa de execução isolada, mas o ambiente de execução disponível para a pesquisa não conseguiu resolver/acessar GitHub para clonar e construir os candidatos. A auditoria de código continuou por GitHub/API, porém isso **não substitui uma execução E2E**.

Teste	Diffusion Studio	OpenReel	Penpot Motion	OpenCut
Arte 2D editável	Parcial forte — objetos básicos e texto; path editor insuficiente	Parcial/forte — shapes/text/SVG, mas não Penpot-class vector	Evidência forte , herdada de Penpot	Parcial
Motion/ keyframes → MP4	Evidência estática forte; não executado	Evidência estática forte; não executado	Experimental	Parcial
NLE vídeo/ áudio	Evidência forte; não executado	Evidência forte; não executado	Não é objetivo principal	Evidência forte para editor clássico
3D na composição	Parcial: exemplo Three.js surface	Parcial/forte: scene3d, glTF, materiais/ câmera	Não	Não é força
Agente modifica projeto editável	Muito forte arquiteturalmente	Forte por tools/project mutations	Forte via MCP/ plugin	MCP é principalmente direção futura
Reabrir + preview/ export parity	Arquitetura favorável, não validado E2E	Serializer favorável, não validado E2E	Experimental	não validado

A distinção “evidência estática forte” versus “teste aprovado” é intencional.

Comparação técnica e matriz de decisão

Matriz dos candidatos

Os pesos são exatamente os especificados na missão. As notas representam **estado efetivamente observado em outubro de 2026**, sem dar crédito a roadmap como implementação.

Candidato	Implementado 25%	IA 15%	Projeto unificado 20%	Velocidade 15%	Arquitetura 10%	Manutenção 10%	Licença 5%	Total
Diffusion Studio	7,2	9,5	9,5	8,2	9,5	5,0	7,0	8,15
OpenReel	8,5	8,5	7,6	8,7	7,5	5,7	10,0	8,04
Penpot Motion	6,8	8,0	5,5	5,5	7,8	6,0	7,0	6,55
Editor Hermes do zero sobre libs	3,5	9,0	9,0	3,0	9,5	6,0	9,0	6,47
OpenCut atual/ reescrita	6,5	3,0	4,5	5,5	8,0	5,0	10,0	5,60

Diffusion Studio — 8,15. A nota extraordinária em IA/unificação vem de uma característica concretamente documentada: source é documento, IDs conectam editor/runtime/source e edits visuais escrevem de volta ao source. A penalização em “implementado” e “manutenção” decorre de sua juventude e lacunas vetoriais.

OpenReel — 8,04. Ganha em funcionalidades imediatamente utilizáveis e licença MIT. Perde para Diffusion em pureza/uniformidade do documento e round-trip code-as-document.

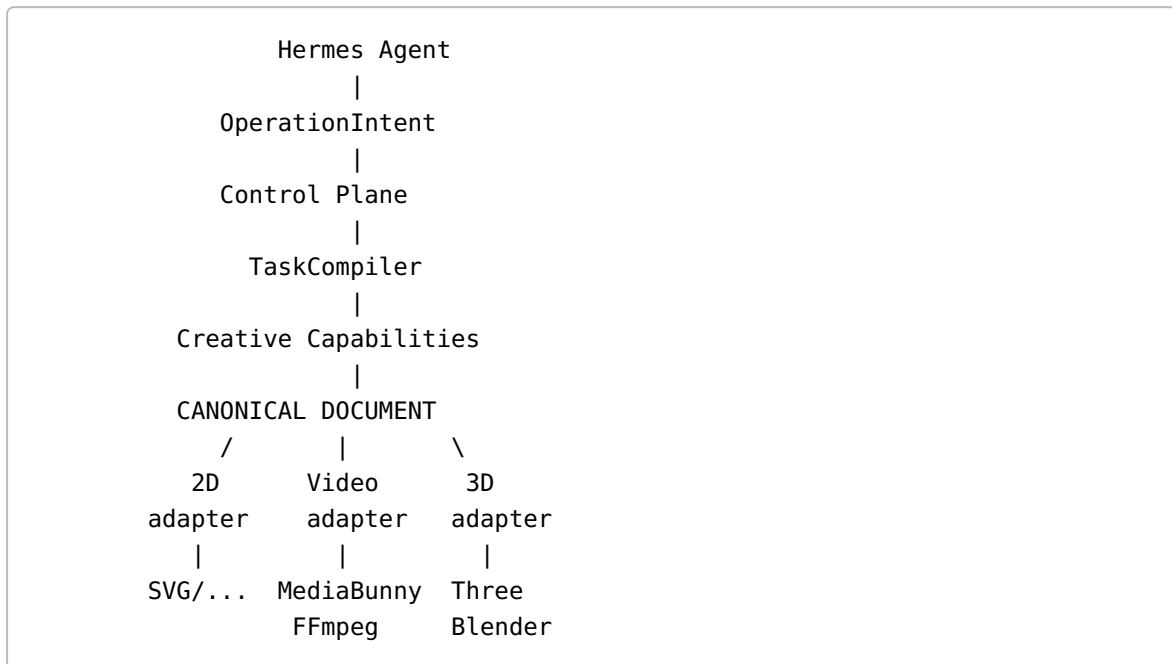
A diferença de **0,11** é pequena demais para justificar uma decisão irreversível sem o POC E2E.

Matriz das estratégias arquiteturais

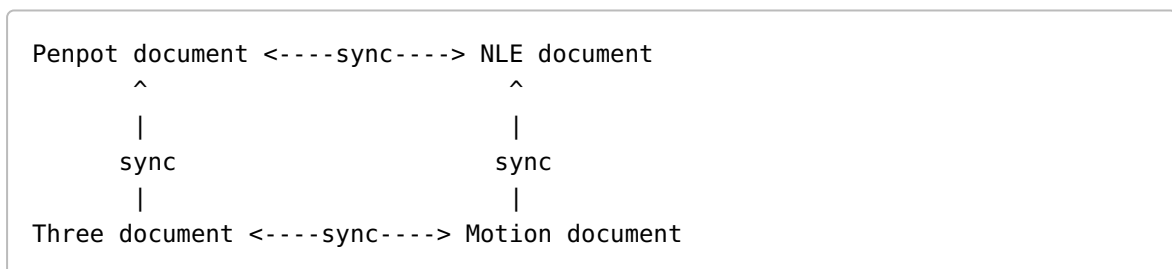
Estratégia	Score ponderado	Avaliação
Arquitetura híbrida canônica: núcleo reutilizado + motores especializados	8,32	Recomendada
Adaptar profundamente um editor	7,70	Boa se o POC revelar um vencedor inequívoco
Editor próprio apenas com bibliotecas	6,53	Muito trabalho redundante
Aplicações independentes sincronizadas	6,48	Rápida para demos, fraca para documento unificado

A diferença entre “híbrida” e “apps sincronizados” é fundamental:

Híbrida recomendada



Não recomendado



A segunda topologia cria conflitos de identidade e semântica que crescem combinatoriamente.

Sensibilidade

Se a prioridade máxima é “ter resultados de vídeo profissionais utilizáveis o mais rápido possível”: **OpenReel passa a ser o vencedor.** Ele oferece mais NLE convencional pronto, MIT e menos incerteza sobre features essenciais de vídeo.

Se a prioridade é “AI-first, humano e agente modificam exatamente o mesmo projeto”: **Diffusion Studio vence claramente.**

Se a prioridade é design estático sofisticado: Penpot vence o subsistema, mas não o workstation.

46

Se a prioridade é arquitetura de media kernel em Rust: OpenCut é uma referência muito melhor que sua classificação geral sugere; seu tempo racional e compositor wgpu são ideias que devem ser absorvidas. 37

Se a prioridade é licença permissiva: OpenReel/Three/Fabric/Konva/MediaBunny ganham.

Recomendação final

A resposta técnica às dez perguntas centrais é:

Questão	Conclusão
Qual projeto é mais vantajoso como base?	Diffusion Studio Editor é o primeiro candidato para base arquitetural; OpenReel é o primeiro candidato para base de produto imediato. A decisão entre os dois deve depender do POC, não de README.
Existe fork superior ao original?	Não encontrei um fork genericamente superior a OpenReel. O fork ComfyUI é melhor especificamente para embedding/AI workflow. <code>girafic/penpot</code> é superior ao Penpot upstream apenas como experimento de motion, não como plataforma geral.
Quais libs adicionar?	Three.js/R3F; MediaBunny; FFmpeg; SAM2; faster-whisper. Fabric/Penpot somente se o gap vetorial exigir. Blender como capability externa.
Melhor integração IA?	Agente não deve “pintar pixels”; deve executar operações tipadas sobre IDs estáveis do documento, passando pelo TaskCompiler/OperationalCapability. Para transformações complexas, pode escrever/patchar source/AST, mas a mutação continua sujeita à autoridade, verifier e readback do Hermes.
Como preservar o mesmo arquivo?	Uma autoridade canônica de composição com stable IDs, timeline/keyframes e referências a native assets. Não converter Blender/Penpot inteiro para JSON genérico.
O que já existe?	Quase todo NLE básico, keyframes, timeline, text/shapes, encode/decode, 3D rendering e IA especializada já existem. A grande lacuna é integrar isso em um contrato coerente e completar autoria vetorial/3D first-class.
Menor trabalho para resultados utilizáveis?	OpenReel.
Melhor potencial de longo prazo?	Arquitetura híbrida baseada no padrão Source-as-Doc/Document/ECS do Diffusion Studio, sob o runtime Hermes.
O que não desenvolver?	Decoder/encoder próprio; engine 3D próprio; segmentador próprio; speech reconhecer próprio; Bézier geometry kernel novo sem antes avaliar Penpot/Fabric/Paper; workflow AI graph próprio em lugar de ComfyUI; DCC próprio em lugar de Blender.
O que falta provar?	Os seis POCs E2E, especialmente round-trip, parity preview/export, performance 4K, Windows/Electron embedding, licensing transitivo e manutenção de IDs quando o agente realiza refactors de source.

A combinação que considero mais inteligente

Camada Hermes

Manter:

- Electron/React Desktop;
- Hermes Agent;
- OperationIntent;
- Control Plane;
- TaskCompiler;
- ArtifactStore;
- OperationalCapabilityRegistry;
- Experience Compiler;
- Journal/receipts/verifiers.

Essa direção já está prescrita pela própria arquitetura Creative Workstation atual.

Creative core candidato

Primeira prova:

```
diffusionstudio/editor/packages/runtime
diffusionstudio/editor/packages/jsx
diffusionstudio/editor/packages/reconciler
diffusionstudio/editor/packages/assets
diffusionstudio/editor/packages/encoder
```

Não recomendo inicialmente substituir o Desktop Hermes pelo Electron shell do Diffusion. A integração deve conservar Hermes como dono do processo e do lifecycle.

Fallback de curto prazo / donor

OpenReel:

- timeline;
- media project model;
- Motion Creator;
- conventional inspector;
- color/audio;
- project serializer;
- Agent tool patterns;
- export patterns.

A licença MIT torna estudo e reaproveitamento seletivo especialmente favoráveis.

Design profissional

Penpot não deve ser o kernel do workstation inteiro. Deve existir inicialmente como:

```
Penpot ↔ MCP/plugin ↔ Hermes
      ↳ SVG/assets ↗
```

Depois, recursos realmente necessários podem ser portados/adaptados ao documento canônico.

Três dimensões

```
Hermes composition node
|
Three.js/R3F scene
|
GLTF
|
Blender capability
```

Blender trabalha como DCC/headless para tarefas que não fazem sentido recriar no editor web.

Vídeo

```
interactive:
MediaBunny + WebCodecs

GPU composition:
Canvas/WebGL/WebGPU

compatibility/export:
FFmpeg
```

IA

```
Hermes Capability
|
+-- segment → SAM2
+-- remove background → rembg / selected segmentation pipeline
+-- transcribe → faster-whisper
+-- source separation → audited Demucs pipeline
+-- generate assets → ComfyUI
+-- generate 3D → separately admitted model
+-- DCC operation → Blender bpy
```

O output de todos esses processos volta como **asset/mask/keyframes/nodes editáveis**, nunca apenas como um MP4 achatado, salvo quando o usuário pede render final.

Hipótese que eu rejeitaria

Eu **não recomendo transformar Penpot profundamente no Creative Workstation inteiro**.

Isso aproveitaria o melhor editor 2D da lista, mas obrigaria Hermes a construir dentro de uma stack grande e especializada tudo aquilo que Penpot não é: NLE, media decoding, audio timeline, frame-

accurate export e 3D compositing. O fork Motion prova que animação é possível; ele não prova que Penpot seja a abstração correta para Premiere + After Effects + Blender. ⁴⁷

Também **não recomendaria desenvolver agora React + Konva/Fabric + timeline + FFmpeg inteiramente do zero**. Isso descartaria justamente as partes difíceis já resolvidas por Diffusion/OpenReel.

E **não recomendaria quatro miniaplicativos independentes com sincronização bidirecional**. Essa abordagem é boa para integração de ferramentas, mas ruim como modelo de documento.

Plano de implementação

Plano de resultado imediato

O objetivo aqui é colocar Hermes produzindo artes e vídeos reais sem condicionar o progresso à criação do workstation ideal.

Primeiro gate: benchmark executável Diffusion Studio versus OpenReel.

Não fazer integração estrutural antes desse benchmark.

Criar o mesmo fixture nos dois candidatos:

```
1920×1080 @ 30 fps
```

```
image  
text  
vector/shape  
group  
video A  
video B  
audio  
mask  
three.js/glTF layer  
keyframed transform  
transition  
nested composition
```

O Hermes Agent deve realizar pelo menos:

```
rename layer  
move text  
change text  
trim video  
split clip  
change color  
create 3 keyframes  
change easing
```

```
add 3D object
modify camera
export
save
close
reopen
modify again
```

Critério de conclusão: cada alteração deve continuar manualmente editável.

Nenhuma operação vale se o agente simplesmente gerar um novo MP4.

Segundo gate: integridade.

Para cada projeto:

```
frame preview at t = 0
frame preview at t = 1.000
frame preview at t = 3.500
    ↓
export
    ↓
decode same frames
    ↓
pixel/perceptual comparison
```

Também comparar:

- timeline structure antes/depois;
- stable IDs;
- missing assets;
- keyframe values;
- font resolution;
- audio duration;
- frame count.

Esse gate provavelmente é mais importante que qualquer comparação de screenshots de UI.

Terceiro gate: integração mínima Hermes.

O vencedor não recebe autoridade própria.

Criar capabilities conceituais do tipo:

```
creative.project.open
creative.project.inspect
creative.node.create
creative.node.patch
```

```
creative.node.delete
creative.timeline.trim
creative.timeline.split
creative.keyframe.set
creative.asset.import
creative.frame.capture
creative.project.validate
creative.render.export
```

Cada uma deve mapear para `OperationalCapability` com input/output schema, effect, route, verificar e provenance, aproveitando a infraestrutura já existente.

Quarto gate: vídeo utilizável.

Integrar FFmpeg somente na boundary em que é necessário:

- probing/fallback;
- import incompatível;
- final codec;
- image/audio sequence;
- proxy generation.

Não transformar FFmpeg commands em source of truth do projeto.

Quinto gate: IA imediatamente útil.

Adicionar, nessa ordem:

SAM2 → masks/rotoscopia; ⁴⁸
faster-whisper → captions/transcript; ⁴⁹
background removal; ⁵⁰
ComfyUI external generation → import returned assets. ³⁰

Cada resultado precisa entrar no documento como entidade/manipulável.

Critério de saída do plano imediato:

Hermes recebe um prompt como:

“Use este vídeo, corte as pausas, faça versão 9:16, remova o fundo da apresentadora, adicione título animado, coloque este GLB à direita, música abaixo da voz e exporte MP4.”

e produz um projeto que o usuário pode abrir e continuar editando manualmente.

Plano de evolução

Após provar o vertical slice, o foco muda de “funciona” para “Creative Workstation unificado”.

Consolidar a Creative Document API.

Não começar pela serialização física. Começar pelos invariantes:

```
StableNodeId  
StableAssetId  
CompositionId  
timebase  
parent/child  
property addressing  
animation tracks  
render order  
effects  
masks  
source references  
capability requirements  
schema/minimum reader version
```

OpenReel já demonstra version/capability/minimum-reader e Diffusion demonstra stable source identities; as duas ideias devem convergir.

Introduzir timebase determinístico.

Usar integer ticks + rational frame rate no contrato do Hermes, adaptando o padrão comprovado conceitualmente no OpenCut. ³⁷

A regra deve ser:

```
tempo autoritativo ≠ JavaScript floating wall clock
```

Elevar 3D de Surface para first-class node.

Inicialmente:

```
<surface renderer="three">
```

Depois:

```
<scene3d>  
  <camera .../>  
  <light .../>  
  <model asset="x.glb">  
    <keyframes .../>  
  </model>  
</scene3d>
```

OpenReel já contém um vocabulário útil para esse desenho; não é necessário inventar do zero os conceitos.

Fechar o gap vetorial.

Antes de escrever uma pen tool nova, comparar três caminhos em código:

- portar/adaptar partes de Fabric/Paper;
- integrar Penpot como authoring backend;
- implementar `path` sobre o renderer já adotado.

Só depois dessa prova decidir.

O que não faz sentido é portar o Penpot inteiro para resolver uma pen tool.

Graph editor.

Estudar e adaptar os invariantes de:

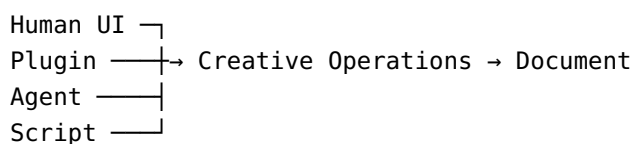
- Penpot Motion;
- OpenCut graph editor;
- Theatre.js.

Não introduzir uma segunda animation engine como autoridade.

Plugin API.

Plugins não devem mutar stores internos diretamente. Expor a mesma Creative Document Operations API usada pelo Hermes Agent.

Isso produz:

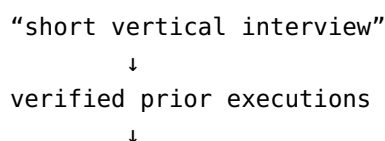


em vez de quatro caminhos internos diferentes.

Experience Compiler.

Operações criativas bem-sucedidas podem então ser convertidas em capabilities/procedures reutilizáveis pelo sistema existente. O repositório Hermes já possui justamente o lifecycle e promotion machinery necessários.

Isso permite, por exemplo:



```
reusable creative capability
↓
cheap deterministic execution
```

sem criar um segundo sistema de skills.

Gradual Rust/WASM adoption.

Não reescrever o editor em Rust.

Mover para Rust/WASM apenas o que profiling justificar:

- timeline math;
- geometry;
- masks;
- raster kernels;
- compositing;
- waveform;
- color transforms;
- codecs quando apropriado.

OpenCut demonstra que essa divisão é viável. 37

Ordem recomendada de decisão

A sequência que minimiza arrependimento técnico é:

Primeiro: POC Diffusion Studio.

Em paralelo: POC OpenReel com o mesmo fixture.

Depois: escolher o creative runtime/editing core com base em round-trip, export parity e Windows embedding — não em feature count.

Usar Penpot: como editor/design adapter e referência de vetores, não como kernel universal.

Usar Three.js: como renderer 3D primário.

Usar Blender: como DCC externo/headless.

Usar MediaBunny/WebCodecs: para o loop interativo.

Usar FFmpeg: como compatibilidade e encoder final.

Usar modelos locais: como OperationalCapabilities isoladas.

Evoluir: uma única Creative Document authority sob o Hermes TaskCompiler/Control Plane.

A conclusão, portanto, não é “escolher um aplicativo”. É:

O caminho de menor redundância é reutilizar um editor/runtime AI-native existente como núcleo de composição, manter Hermes como autoridade operacional, adicionar motores especializados por adapters e fazer do projeto criativo — não dos renderizadores — a unidade universal.

Com a evidência disponível em **9 de outubro de 2026**, **Diffusion Studio** é a arquitetura mais promissora para esse núcleo; **OpenReel** é a alternativa de menor risco para entregar funcionalidades imediatamente; **Penpot** é o melhor subsistema de design, mas não a melhor fundação do workstation inteiro; **OpenCut** é uma excelente fonte de ideias de kernel, ainda não a base mais pronta.

A decisão final entre **Diffusion Studio-based hybrid** e **OpenReel-based hybrid** permanece corretamente condicionada a uma única dívida de evidência importante: executar os seis testes end-to-end, porque a diferença ponderada entre ambos é pequena e esta investigação, embora tenha validado extensivamente a estrutura real do código, **não teve um ambiente capaz de construir e rodar os repositórios para certificar runtime, performance e preview/export parity.**

1 33 46 Penpot: The open-source design platform for Product ... - GitHub

<https://github.com/penpot/penpot>

2 Fabric.js Javascript Library

<https://www.fabricjs.com/>

3 Konva.js is an HTML5 Canvas JavaScript framework ... - GitHub

<https://github.com/konva/konva>

4 svg.js/README.md at master · svgdotjs/svg.js · GitHub

<https://github.com/svgdotjs/svg.js/blob/master/README.md>

5 paper.js/README.md at develop · paperjs/paper.js · GitHub

<https://github.com/paperjs/paper.js/blob/develop/README.md>

6 GitHub - pixijs/pixijs: The HTML5 Creation Engine

<https://github.com/pixijs/pixijs>

7 excalidraw/README.md at master - GitHub

<https://github.com/excalidraw/excalidraw/blob/master/README.md>

8 tldraw/LICENSE.md at main - GitHub

<https://github.com/tldraw/tldraw/blob/main/LICENSE.md>

9 GitHub - GraphiteEditor/Graphite: Community-built ...

<https://github.com/GraphiteEditor/Graphite>

10 remotion-dev/remotion: Make videos programmatically with React

<https://github.com/remotion-dev/remotion>

11 midrender/revideo: Create Videos with Code - GitHub

<https://github.com/midrender/revideo>

12 motion-canvas/motion-canvas: Visualize Your Ideas With Code

<https://github.com/motion-canvas/motion-canvas>

13 theatre-js/theatre: Motion design editor for the web - GitHub

<https://github.com/theatre-js/theatre>

- 14 **GitHub - motiondivision/motion: A modern animation library for ...**
<https://github.com/motiondivision/motion>
- 15 **GSAP (GreenSock Animation Platform), a JavaScript ... - GitHub**
<https://github.com/greensock/gsap>
- 16 **juliangarnier/anime: JavaScript animation engine - GitHub**
<https://github.com/juliangarnier/anime>
- 17 **airbnb/lottie-web: Render After Effects animations ... - GitHub**
<https://github.com/airbnb/lottie-web>
- 18 40 **mrdoob/three.js: JavaScript 3D Library. - GitHub**
<https://github.com/mrdoob/three.js/>
- 19 **R3F - Introduction - React Three Fiber - Poimandres**
<https://r3f.docs.pmnd.rs/getting-started/introduction>
- 20 **Babylon.js - GitHub**
<https://github.com/BabylonJS/Babylon.js/wiki/00-Babylon.js-Primer>
- 21 **Rendering From The Command Line - Blender 5.2 LTS Manual**
https://docs.blender.org/manual/en/latest/advanced/command_line/render.html
- 22 45 **FFmpeg License and Legal Considerations**
<https://www.ffmpeg.org/legal.html>
- 23 39 **Introduction | Mediabunny**
<https://mediabunny.dev/guide/introduction>
- 24 **MLT framework**
<https://www.mltframework.org/>
- 25 48 **GitHub - facebookresearch/sam2: The repository provides code ...**
<https://github.com/facebookresearch/sam2>
- 26 50 **Rembg is a tool to remove images background - GitHub**
<https://github.com/danielgatis/rembg>
- 27 **facebookresearch/co-tracker: CoTracker is a model for tracking ...**
<https://github.com/facebookresearch/co-tracker>
- 28 **hzwer/Practical-RIFE: More practical frame interpolation ...**
<https://github.com/hzwer/practical-rife>
- 29 **Robust Video Matting in PyTorch, TensorFlow ... - GitHub**
<https://github.com/PeterL1n/RobustVideoMatting>
- 30 **GitHub - Comfy-Org/ComfyUI: The most powerful and modular ...**
<https://github.com/comfyanonymous/ComfyUI?via=nexttoolai>
- 31 **microsoft/TRELLIS: Official repo for paper "Structured 3D ...**
<https://github.com/microsoft/TRELLIS>
- 32 **Hunyuan3D-2/LICENSE at main · Tencent-Hunyuan ... - GitHub**
<https://github.com/Tencent-Hunyuan/Hunyuan3D-2/blob/main/LICENSE>
- 34 **penpot/mcp/README.md at develop - GitHub**
<https://github.com/penpot/penpot/blob/develop/mcp/README.md>
- 35 47 **Penpot Motion: keyframe animation for boards (experimental)**
<https://community.penpot.app/t/penpot-motion-keyframe-animation-for-boards-experimental/10914>

36 38 README.md - OpenCut-app - GitHub

<https://github.com/OpenCut-app/OpenCut/blob/main/README.md>

37 Releases · OpenCut-app/OpenCut - GitHub

<https://github.com/OpenCut-app/OpenCut/releases>

41 MPL 2.0 FAQ - Mozilla

<https://www.mozilla.org/en-US/MPL/2.0/FAQ/>

42 GNU Lesser General Public License v3.0

<https://www.gnu.org/licenses/lgpl-3.0.html>

43 Frequently Asked Questions about the GNU Licenses

<https://www.gnu.org/licenses/gpl-faq.html>

44 GNU Affero General Public License - Free Software Foundation

<https://www.gnu.org/licenses/agpl-3.0.html>

49 Faster Whisper transcription with CTranslate2 - GitHub

<https://github.com/SYSTRAN/faster-whisper>